From Algorithms to Articulation

Relational pseudocode as arguments rendered operational

Abstract

As software production becomes increasingly mediated by large language models, the central educational problem shifts from the manual production of code towards the articulation and judgement of the ideas that code implements. Robert Pirsig's account of the motorcycle provides a useful starting point. Technical decomposition renders a machine inspectable, yet does not exhaust its meaning or determine the Quality of an intervention. This essay applies that distinction to computer science education. It argues that pseudocode occupied a valuable intermediate position between natural language and executable syntax, and that its decline is poorly timed as software engineering moves towards specification, retrieval, knowledge graphs and model-generated implementation. Procedural thinking remains essential, but students increasingly require relational, evidential and interpretative forms of reasoning. The essay concludes with an appeal for software engineers to engage more directly with school learning materials, helping students learn how to specify purposes, represent domains, judge generated systems and recognise the difference between code that runs and a solution that fits.

The motorcycle that analysis leaves behind

In *Zen and the Art of Motorcycle Maintenance*, Pirsig performs an exemplary act of technical decomposition. The motorcycle is divided first by components, then by functions. Components divide into assemblies and subassemblies; functions divide into cycles, operations and causal sequences. The analysis is rigorous. It shows how rationality converts a complicated object into a hierarchy that can be named, inspected and repaired.

Yet Pirsig immediately marks the limit of this description which is nearly impossible to understand unless one already knows how a motorcycle works. Pirsig condenses his point in four short formulations. The machine is a "system of concepts worked out in steel"; it is "primarily a mental phenomenon"; beneath the visible system lies "rationality itself"; and "Quality isn't a method".[1] He elaborates the motorcycle's materiality without fully exposing how thoroughly the material machine has been shaped by distinctions made in thought. His archetypal machine part a tappet, that small part of an internal-combustion engine's valve mechanism which transfers the motion of the camshaft to the valve, either directly or through a push-rod and rocker arm. Such a tappet must maintain a carefully specified clearance, existing within a designed account of what an engine is, what a valve train does, which motions matter, what counts as tolerance and what counts as failure.

[1] Robert M. Pirsig, *Zen and the Art of Motorcycle Maintenance: An Inquiry into Values* (1974). The linked scan supplied for this essay is available at `https://archive.org/details/zenandtheartofmotorcyclemaintenancerobertpirsigm._833_V/page/13/mode/2up`.

All is to say that Pirsig appears useful for describing our attitude towards software engineering. As the motorcycle can be divided into components and functions, but the resulting hierarchy does not ride so the program can be reduced to instructions, but the instructions do not reveal what the program means to a human in context. Code though less substantial than steel is a materialisation of distinctions. The specification of a class, function, table, schema, prompt or ontology is just as much a decision about what exists, what belongs together and what may be ignored as contained in any Haynes workshop manual.

An engineer more than describes a system in envisioning it.

The descent of machine

Traditional programming education presented abstraction as a ladder whose lower rungs remained visible. A programmer might work in Java, C or Python, yet the language was understood as an organised convenience above machine operations. One could, at least in principle, descend:

$$\text{algorithm} \longrightarrow \text{high-level language} \longrightarrow \text{assembly} \longrightarrow \text{machine code}$$

The direction of teaching may or may not have ran that way, but the implied justification ran downward. A loop could be explained through jumps and registers; a variable through storage; a method call through stack frames and addresses. The higher language compressed the lower mechanism without entirely concealing it. The difficulty is that contemporary programming within an AI-assisted development framework, the programmer may specify a desired behaviour while lacking any realistic capacity to descend through the generated solution down to basic implementation. Libraries already made complete descent difficult; generated code and model-mediated systems can make it practically impossible. The abstraction ladder has become a curtain. The programmer/engineer must increasingly judge systems whose internal operations cannot be fully surveyed. Verification, provenance, architecture and purpose therefore become more important at the same moment that direct authorship becomes less complete.

This reductive picture had pedagogical and measurable power, making correctness inspectable. The student learned that an apparently fluent program ultimately became precise operations upon a finite machine. *Pseudocode* occupied a valuable position within this arrangement. It abstracted from the incidental syntax of a particular language while preserving sequence, branching, iteration, state and termination. It was a language of processes, neither executable code nor unconstrained prose.

Contentiously, the revised IB Diploma Computer Science Programme course moves away from language-independent pseudocode in favour of explicit Java or Python. That may improve executability and make student work easier to test, but it also risks confusing syntactic correctness with computational understanding. As machines become increasingly capable of generating valid code, the educational priority may need to move in the opposite direction: towards the disciplined formulation, interrogation and validation of the idea before its implementation. Pseudocode mattered because it occupied that intermediate space. It required more precision than ordinary language while refusing to let the grammar of a particular programming language become the thought itself.[2]

[2]The revised IB Diploma Programme Computer Science course was launched in 2025 for first assessment in

There is, however, a less charitable possibility. Pseudocode may have become awkward for examination systems precisely because it required *qualitative judgement*. Its strength was also its inconvenience: an examiner had to decide whether a process was sufficiently clear, coherent and computationally exact without relying entirely upon compilation or rigid syntactic conformity.[3] Examination boards are driven towards mensuration by cost, scale and the desire for clean deliberation. What can be counted, standardised and rapidly moderated is institutionally safer than what must be judged.

The irony is that large language models return us to the very difficulty such systems tried to remove. When syntactically valid code can be produced cheaply and in abundance, the scarce capacity is no longer generation but discrimination. We are left once again with judgement: whether the idea is sound, whether the process is fitting, whether the representation is intelligible and whether the result has *Quality*. Quantity becomes easy. Quality becomes the work. In that sense, the educational priority may need to reverse.

For what reason

Just as ball bearings are spherical *for a reason*, their form guided by a mind and a purpose, the Earth is spherical *from causes*, without having been designed to be so.[4] Dennett's distinction as such between asking how something *came to be* versus asking (the profounder) *what it is for* could serve as emblematic of the changing role of the software engineer.

Older computer science curricula were organised around the first question: how does the program work, how does the algorithm proceed, how is the machine instructed? Those questions remain indispensable. Yet as implementation becomes easier to delegate, the more prescient question becomes: for what reason is this system being built, what conception of the user does it contain, what purposes has it mistaken for requirements, and what would count as a good result?

This is where software engineers have something important to feed-back and contribute to school education. They can help students see that programming is more than the production of executable instructions, being rather the disciplined translation of human purposes into systems that offer up useful*affordances* to other people.

Tapping into analogies

Pirsig's mechanic can remove a cover, measure a clearance and follow a causal chain through the engine as the machine is ultimately inspectable. Even when the full articulation of its hierarchy, as set out in a Haynes workshop manual, does not capture the lived motorcycle, the relation between component and function remains comparatively stable.

2027. It replaces the former language-independent pseudocode used in assessment with programming in Java or Python.

[3]Tellingly, IB pseudocode was itself highly prescriptive: its conventions were sufficiently regular that tools could translate it into executable languages and run the resulting code. The student-built *IbPseudo* platform, for example, translates IB pseudocode into Python or JavaScript and states that generated code follows the pseudocode rules prescribed by the IB. See `https://ibpseudo.vercel.app/`. The irony is that even a notation designed to sit between natural language and executable code was made increasingly interpreter-like in order to render judgement more standardisable.

[4]This distinction follows Daniel Dennett's contrast between explaining an artefact from the *design stance* and explaining a natural object through its causal history. See Daniel C. Dennett, *The Intentional Stance* (Cambridge, MA: MIT Press, 1987), and *Darwin's Dangerous Idea* (London: Penguin, 1995).

Modern software as the author envisions it, is less obliging. Its behaviour may emerge from *distributed services*, learned parameters, external databases and shifting dependencies.No single engineer necessarily possesses the whole causal account. In this respect, modern software resembles Leonard Read's humble pencil, popularised by Milton Friedman: the functioning whole depends upon dispersed knowledge that no participant individually commands. Software goes further, however. Its dependencies, learned parameters, external services and retrieved information may leave the causal path from instruction to outcome only partially reconstructable, even by the engineers maintaining it.[5]

From algorithmic to relational language

Classical pseudocode asks a procedural question:

> What sequence of operations transforms this input into the required output?

That question remains indispensable. Yet databases, knowledge graphs and retrieval-augmented generation introduce another family of questions:

> What entities are present? How are they related? Which claims are supported? What context limits the inference? Which evidence should be retrieved?

These questions require a language of ideas and relations as well as a language of processes. A school-level pseudocode might therefore contain both procedural and declarative forms:

```
ENTITY Tappet
ENTITY Engine
Tappet PART-OF Engine

CLAIM ExcessiveClearance CAUSES TappingNoise
SUPPORTED-BY ServiceManual.Section4
APPLIES-WHEN Engine = Cold

FIND probable_cause
WHERE symptom = TappingNoise
RETURN claim, evidence, conditions
```

We see that retrieval still requires ranking; databases still require execution plans; graph queries still become machine operations. The educational shift is that the student must learn to formalise not only *what happens next*, but also *what is being claimed*. The central intellectual acts become naming, relating, constraining and justifying.

This gives renewed importance to ontology, semantics and even etymology. Names are not cosmetic labels placed on completed structures. They carry inherited distinctions. A term may gather meanings from law, medicine, commerce or ordinary speech before entering a database schema. Once formalised, those meanings become selectable fields, categories and edges. The machine then reproduces the distinction at scale.

The older engineer could imagine that values entered only after the technical work was

[5]Leonard E. Read, "I, Pencil: My Family Tree as Told to Leonard E. Read," *The Freeman*, December 1958. Milton Friedman later popularised the example as an illustration of dispersed knowledge and coordination in a market economy.

finished. In a knowledge graph or RAG system, values are already present in the ontology: in what is admitted as an entity, what is treated as evidence, which relations are searchable and whose language becomes canonical.

Engineered Triage

The tappet remains a useful illustration of the movement from procedural diagnosis to relational retrieval. A rhythmic tapping sound may be approached through a loop:

```
LISTEN for tapping
CHECK valve clearance
ADJUST tappet if necessary
RESTART engine
REPEAT until tapping stops
```

The governing question is procedural:

> What step comes next?

A knowledge-graph representation begins elsewhere:

```
SYMPTOM: rhythmic tapping
LOCATED-NEAR: cylinder head
VARIES-WITH: engine speed

POSSIBLE-CAUSE: excessive valve clearance
POSSIBLE-CAUSE: worn rocker arm
POSSIBLE-CAUSE: inadequate lubrication

REQUIRES-CONTEXT: engine temperature
REQUIRES-EVIDENCE: service manual
```

Its governing question is relational:

> What connections make this symptom intelligible?

But how is one to grade? Graded on semantic coherence, contextual constraint, or minimization of taxonomic bias? For a classroom task, we could require students to annotate one design choice: identifying one category or relation they considered but rejected, and explaining why.

Assessing Relational Pseudocode

If relational pseudocode is to become pedagogically useful, educators will need criteria by which it can be assessed. The task is not to reduce *Quality* to a checklist, but to distinguish formal weaknesses from stronger representations.

A relational model might be judged across five dimensions:

- **Semantic coherence:** whether entities and relations are used consistently and intelligibly;
- **Contextual constraint:** whether the student states the conditions under which a claim applies;

- **Evidential grounding:** whether claims are linked to appropriate observations, documents or rules;
- **Relational completeness:** whether the important connections are present without unnecessary proliferation;
- **Taxonomic discipline:** whether categories are well chosen, distinctions are preserved and avoidable bias is minimised.

These criteria can make the work inspectable. They allow an examiner to identify unsupported causal claims, missing conditions, confused categories or evidence detached from inference. Yet a final holistic judgement remains necessary:

> Does this representation make the problem more intelligible?

That question cannot be fully decomposed into marks. Two knowledge graphs may contain the same number of entities, relations and citations, while one captures the structure of the problem more fittingly than the other. The rubric measures formal adequacy; the holistic judgement attends to *Quality*.

The resemblance to a philosophical rubric is revealing. Relational pseudocode is an argument rendered operational: a set of claims about what exists, how things are connected, what counts as evidence and which inferences are licensed. Its novelty lies in requiring those philosophical commitments to become executable.

Bodily functions

At this point the motorcycle metaphor starts to show its limitations. A detour into the orders of magnitude more complex system that is a human presenting symptoms to an A&E should still be humbling to the most sophisticated engineer[6] An organ can be isolated anatomically and assigned a function, yet its actual operation depends upon circulatory, hormonal, neural, microbial and environmental relations. The heart is a pump, but that description is radically insufficient as an account of circulation, exertion, fear or life. Form and function remain distinguishable, though neither is intelligible in isolation[7].

Consider two patients arriving at an accident and emergency department with abdominal pain, one woman and one man. A triage protocol may begin identically for both:

```
RECORD pain severity
MEASURE pulse, temperature and blood pressure
CHECK for guarding or rigidity
ESCALATE if signs of instability are present
```

[6]The value of our biological holism analogy is in reminding us that decomposition does not guarantee explanation. A list of organs does not amount to physiology, and a list of functions does not amount to a person. Likewise, a repository (being the stored collection of a software project's files, typically its source code, configuration files, documentation, tests, and version history) or codebase does not by itself disclose the working meaning of a software system.

[7]The analogy with bodily organs points beyond straightforward mechanical decomposition. Denis Noble argues that the organism is not merely a vehicle constructed and directed by its genes: causation operates across molecular, cellular, organ and organismal levels. The score does not contain the performance, nor can the music be reduced to its individual notes. See Denis Noble, *The Music of Life: Biology Beyond Genes* (Oxford: Oxford University Press, 2006).

The protocol is indispensable, but the symptom acquires meaning only within a wider network of relations: location, onset, pregnancy possibility, menstrual history, urinary symptoms, vascular risk, previous surgery and referred pain. The same surface complaint does not imply the same diagnostic space.

This is an argument against mistaking the algorithm for the whole act of understanding. The *procedural* system ensures that essential steps are not omitted. The *relational* system retrieves plausible connections, contextual evidence and urgent exclusions. Clinical judgement still determines which relation matters now, which reassuring sign should be distrusted and when the patient does not fit the apparent pattern.

Software systems increasingly resemble this kind of organisation as their parts are more than merely assembled being situated in networks of dependency, contextualized information and interpretation[8]. Much traditional programming education concentrated rather on procedural control:

```
SEQUENCE
SELECT
ITERATE
RETURN
```

Contemporary systems increasingly operate over bodies of data, documents, claims and relationships whose significance cannot be fixed in advance as a single decision tree. Retrieval-augmented systems and knowledge graphs therefore require another mode of thought:

```
IDENTIFY entities
STATE relations
RETRIEVE evidence
APPLY context
RANK plausible interpretations
RETURN answer with provenance
```

The change is in what the engineer is being asked to formalise. The older algorithm presumes that the relevant path has already been specified. The newer system that is our triage nurse must help determine which path is justified by the situation.

Quality Control

This is where Pirsig's notion of *Quality* becomes technically relevant. Our turn to biological systems Pirsig would likely consider as the ultimate synthesis of classic-mechanical and romantic-holistic views of reality. Two systems presenting similar behaviours borne of differing functionality is never the whole story.

Two engineered systems also may execute the same procedures, retrieve the same documentation while differing profoundly in whether they have represented a problem well. One may identify the right entities, preserve provenance and make uncertainty visible. Another may collapse context, conceal assumptions and return an answer with unwarranted confidence. Correct execution does not settle the quality of the conceptual model.

[8] A database field has meaning contingent on who entered it, why it was collected, which categories were available at the time of entry and how later systems retrieve it.

We should measure less of what machines can now produce and cultivate more of what they cannot reliably supply: proportion, relevance, restraint, coherence and the recognition of when a solution is technically correct yet conceptually wrong. Quality over quantity may be too familiar a slogan, but the asymmetry is now real. In computational education, as in thermodynamics, the more significant question may increasingly concern the organisation of possibilities rather than the mere expenditure of effort: entropy rather than energy.

As implementation becomes increasingly automated, the educational emphasis should perhaps move upstream: from producing syntactically valid code towards articulating, testing and defending the ideas that the code instantiates. This is close to David Millard's call for *humanist engineering*: engineering in which interpretation, judgement and the *quality* of intention are treated as constitutive technical concerns rather than as mere decoration applied after the system has been built.[9]

When machines increasingly generate the algorithms, engineers assume greater responsibility for the language of intention, relation and evidence surrounding them. This asks the engineer for precision at the level where contemporary systems are actually being designed. An LLM generated program may be syntactically competent while embodying a poor account of the world. A retrieval system may answer fluently while collapsing testimony, policy, measurement and opinion into undifferentiated text. A knowledge graph may be logically consistent while reproducing crude or historically loaded categories. These are failures of engineering because they are failures in the concepts worked out through the system.

The humanities become relevant here as disciplines of interpretation. History shows how categories were formed and whom they served. Philosophy distinguishes *evidence* from *assertion*, *identity* from *classification* and *description* from *value*. Linguistics examines how meaning shifts with context. Literature trains attention upon voice, ambiguity, motive and what a formal account leaves unsaid. Ethics asks which purposes deserve implementation.

Pirsig's central notion of *Quality* is apposite here. An appreciation of Quality is the *cultivated* capacity to recognise that a technically adequate decomposition may still miss the object it claims to describe[10].

Pirsig asks us to judge what Quality is. When the handlebars of John's BMW begin to slip, the proposed repair is a thin aluminium *shim* inserted beneath the clamp. The shim can be cut from a beer can. John recoils from the suggestion. The material may possess the required thickness, strength and malleability, and it will disappear from sight once the repair is complete, yet it appears unworthy of the motorcycle. It has not been manufactured, packaged and named as a BMW component.

The episode can be translated into an algorithm easily enough:

```
MEASURE the clearance
SELECT aluminium of suitable thickness
CUT a shim to fit the clamp
REMOVE sharp edges
INSERT the shim
```

[9] David Millard, "Nobody Laments Machine Code," *The Stranger's Notebook*, Substack, 2026, https://substack.com/@davidmillard/note/p-206416056?r=ngyda.

[10] An account of the rainbow phenomenon can be separated into wavelengths, but an account in terms of wavelength without an observer or water vapour will miss the mark.

```
TIGHTEN the clamp
TEST the handlebars under load
```

The algorithm is correct but it does not tell us whether the repair has Quality. A carelessly cut strip, forced into place and left with a sharp edge, follows the same sequence as an exact, sympathetic repair. A proprietary shim ordered at considerable expense may satisfy the same specification at unnecessary cost may be reassuring to the insurance broker. Replacing the complete assembly would also solve the problem, but likely needlessly wasteful. The algorithm identifies the process, but judgement identifies the *fittest* solution.

The reader may therefore ask where the Quality resides. Is the beer-can repair inferior because it is improvised, or superior because it recognises the material beneath its label? Does customisation diminish the integrity of the machine, or does the careful adaptation of material to circumstance constitute the very work of the artisan? The answer cannot be compiled from the procedure. It must be seen.

This is the difficulty now returning to software engineering. Generated programs may implement the same stated algorithm, pass the same tests and produce the same visible result. Yet one may be proportionate, intelligible and fitted to its context, while another is brittle, wasteful or conceptually crude. The role of education cannot therefore end with the production of executable code. Students must learn to recognise the difference between a solution that runs and one that fits. The appeal is humanist in a specifically Pirsigian sense. *Quality* here does not mean general excellence or ethical approval. It is the prior, partly ineffable judgement by which an experienced maker or appreciator recognises that one solution is more fitting, coherent or alive than another before that judgement has been reduced to rules. Software engineering therefore requires more than correctness, efficiency and compliance. It requires the cultivated capacity to perceive what is worth building, what form the system should take and when an implementation is technically valid yet somehow wrong.

A revised educational sequence

The educational task is therefore to teach students how to judge whether entities, relations, categories and purposes encoded by a system are *fitting*. Software engineers have a particular contribution to make here. They can help schools move beyond building curricula envisioning programming as wholly algorithmic, from the (mere) production of instructions towards the disciplined articulation of how a problem is to be envisioned and enshrined. Computer science education should therefore resist two symmetrical errors. It should not abandon executable code for vague vibe prompting tutition. Nor should it respond to AI by retreating into syntax as though manual code verification were now the whole discipline.

A more coherent sequence would move repeatedly among four levels:

- **Algorithm:** the sequence of operations and changes of state;
- **Implementation:** the executable language, runtime and machine constraints;
- **Representation:** the entities, relations, schemas and evidence structures through which the problem is expressed;

- **Quality:** the judgement of whether the representation and resulting system are apt, humane and worth sustaining.

Pseudocode remains central, but its scope should widen. Alongside loops, branches and procedures, students should write controlled relational statements, queries, provenance rules and explicit conditions of applicability. They should learn to move from natural-language intention to a formal intermediate representation, then to executable code, and finally back to a human evaluation of the result.

The decisive future skill may be the ability to articulate an intention without (totally) surrendering it: to name the right entities, preserve relevant distinctions, specify relations, while also demanding evidence and recognising where an apparently functioning system is misialigned to its objective.

Against Unsocial Engineering

Software, even its name preserves an older ontology: software was the pliable *ware* supplied to hard machinery, a manufactured article separable from the machine that executed it. Contemporary systems have made that distinction increasingly difficult to sustain. But engineering has always extended beyond machines. Societies were engineered through land, labour, taxation, law and hierarchy long before the word acquired its modern technical sense. Fields were divided, obligations recorded, populations counted and behaviour shaped through institutions designed to make people legible and governable.

Software now performs much of that work at a finer scale and with far less visibility. It classifies, ranks, predicts, recommends, excludes and directs. We are increasingly being engineered through systems whose categories we did not choose, whose purposes we may not see and whose judgements can travel further than any individual official once could.

That makes Millard's call for *humanist engineering* an urgent one. The strongest inheritances of the humanities, shared interpretation, ethical restraint, historical memory, attentiveness to language, sympathy for ambiguity and respect for persons, must enter the design of these systems at the level of architecture, data and purpose. They cannot remain decorative principles added after deployment.

The question facing engineering education is therefore larger than how to produce functioning software. It is how to form engineers capable of recognising the human world their systems will organise, and capable of building into those systems some of humanity's best ways of seeing, judging and living together.

Source note. The discussion of Pirsig's component and functional decomposition follows the motorcycle hierarchy passage in *Zen and the Art of Motorcycle Maintenance.* A searchable transcription is also available at `https://terebess.hu/english/motor2.html`. All extended wording above is paraphrased; only the four brief phrases explicitly marked as quotations reproduce Pirsig's text.